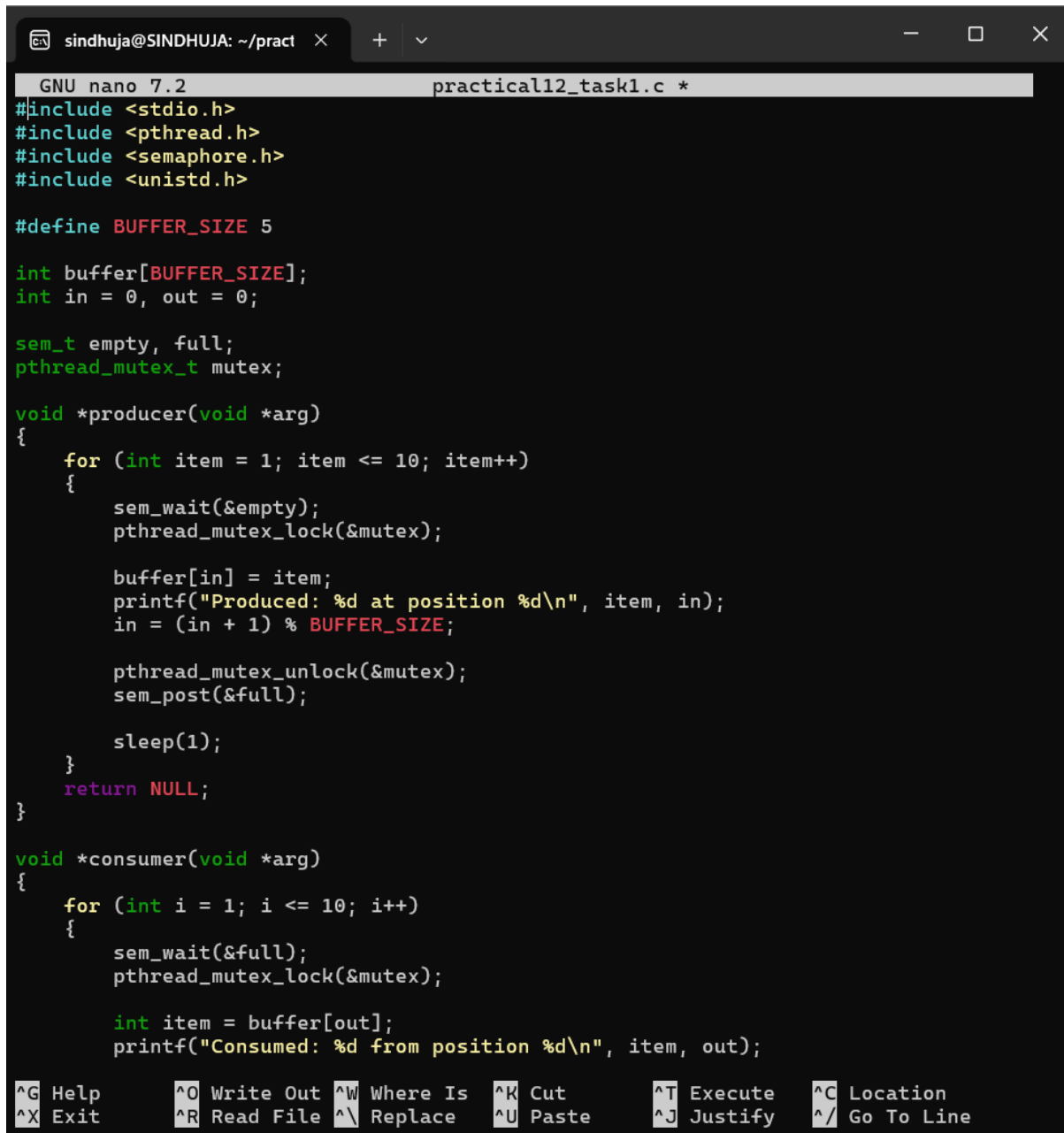


PRACTICAL WEEK 12

Aim:

To implement the Producer–Consumer problem using POSIX threads and counting semaphores, synchronize producer and consumer threads, and verify correct execution using a shared buffer.

Program:



```
GNU nano 7.2 practical12_task1.c *
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int in = 0, out = 0;

sem_t empty, full;
pthread_mutex_t mutex;

void *producer(void *arg)
{
    for (int item = 1; item <= 10; item++)
    {
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);

        buffer[in] = item;
        printf("Produced: %d at position %d\n", item, in);
        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&full);

        sleep(1);
    }
    return NULL;
}

void *consumer(void *arg)
{
    for (int i = 1; i <= 10; i++)
    {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);

        int item = buffer[out];
        printf("Consumed: %d from position %d\n", item, out);
    }
}
```

Help **Write Out** **Where Is** **Cut** **Execute** **Location**
Exit **Read File** **Replace** **Paste** **Justify** **Go To Line**

```

    int item = buffer[out];
    printf("Consumed: %d from position %d\n", item, out);
    out = (out + 1) % BUFFER_SIZE;

    pthread_mutex_unlock(&mutex);
    sem_post(&empty);

    sleep(1);
}
return NULL;
}

int main()
{
    pthread_t producerThread, consumerThread;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&producerThread, NULL, producer, NULL);
    pthread_create(&consumerThread, NULL, consumer, NULL);

    pthread_join(producerThread, NULL);
    pthread_join(consumerThread, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    printf("Producer-Consumer execution completed successfully.\n");

    return 0;
}

```

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location
 ^X Exit ^R Read File ^N Replace ^U Paste ^J Justify ^_ Go To Line

Output:

```

sindhuja@SINDHUJA: ~/pract
sindhuja@SINDHUJA:~$ mkdir practical_week12
sindhuja@SINDHUJA:~$ cd practical_week12
sindhuja@SINDHUJA:~/practical_week12$ nano practical12_task1.c
sindhuja@SINDHUJA:~/practical_week12$ gcc practical12_task1.c -o practical12_task1 -pthread
sindhuja@SINDHUJA:~/practical_week12$ ./practical12_task1
Produced: 1 at position 0
Consumed: 1 from position 0
Produced: 2 at position 1
Consumed: 2 from position 1
Produced: 3 at position 2
Consumed: 3 from position 2
Produced: 4 at position 3
Consumed: 4 from position 3
Produced: 5 at position 4
Consumed: 5 from position 4
Produced: 6 at position 0
Consumed: 6 from position 0
Produced: 7 at position 1
Consumed: 7 from position 1
Produced: 8 at position 2
Consumed: 8 from position 2
Produced: 9 at position 3
Consumed: 9 from position 3
Produced: 10 at position 4
Consumed: 10 from position 4
Producer-Consumer execution completed successfully.
sindhuja@SINDHUJA:~/practical_week12$

```

Aim:

To demonstrate a deadlock scenario using POSIX threads and mutex locks, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

Program:



```
GNU nano 7.2 practical12_task2.c *
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *thread1(void *arg)
{
    printf("Thread 1: Waiting for Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 1: Resource 1 acquired\n");
    sleep(1);

    printf("Thread 1: Waiting for Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 1: Resource 2 acquired\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2(void *arg)
{
    printf("Thread 2: Waiting for Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 2: Resource 2 acquired\n");
    sleep(1);

    printf("Thread 2: Waiting for Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 2: Resource 1 acquired\n");

    pthread_mutex_unlock(&resource1);
    pthread_mutex_unlock(&resource2);

    return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    return 0;
}
```

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark
^X Exit ^R Read File ^_ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo M-G Copy

```
sindhuja@SINDHUJA:~/practical_week12$ nano practical12_task2.c
```

```
sindhuja@SINDHUJA:~/practical_week12$ gcc practical12_task2.c -o practical12_task2 -pthread
```

```
sindhuja@SINDHUJA:~/practical_week12$ ./practical12_task2
```

```
Thread 1: Waiting for Resource 1
```

```
Thread 1: Resource 1 acquired
```

```
Thread 2: Waiting for Resource 2
```

```
Thread 2: Resource 2 acquired
```

```
Thread 1: Waiting for Resource 2
```

```
Thread 2: Waiting for Resource 1
```

```
^C
```



sindhuja@SINDHUJA: ~/pract



GNU nano 7.2

practical12_prevention.c *

```
#include <stdio.h>
#include <pthread.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *worker(void *arg)
{
    int id = *(int *)arg;

    printf("Thread %d: Waiting for Resource 1\n", id);
    pthread_mutex_lock(&resource1);

    printf("Thread %d: Resource 1 acquired\n", id);

    printf("Thread %d: Waiting for Resource 2\n", id);
    pthread_mutex_lock(&resource2);

    printf("Thread %d: Resource 2 acquired\n", id);

    printf("Thread %d: Using resources\n", id);

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

int main()
{
    pthread_t t1, t2;
    int id1 = 1, id2 = 2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    pthread_create(&t1, NULL, worker, &id1);
    pthread_create(&t2, NULL, worker, &id2);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    printf("Deadlock prevented using resource ordering.\n");

    return 0;
}
```

```
sindhuja@SINDHUJA:~/practical_week12$ nano practical12_prevention.c
sindhuja@SINDHUJA:~/practical_week12$ gcc practical12_prevention.c -o practical12_prevention -pthread
sindhuja@SINDHUJA:~/practical_week12$ ./practical12_prevention
Thread 1: Waiting for Resource 1
Thread 1: Resource 1 acquired
Thread 1: Waiting for Resource 2
Thread 1: Resource 2 acquired
Thread 1: Using resources
Thread 2: Waiting for Resource 1
Thread 2: Resource 1 acquired
Thread 2: Waiting for Resource 2
Thread 2: Resource 2 acquired
Thread 2: Using resources
Deadlock prevented using resource ordering.
sindhuja@SINDHUJA:~/practical_week12$ |
```

Observation:

1. Two mutex resources were used by two threads.
2. A deadlock occurred when threads acquired resources in different orders.
3. The program stopped because both threads waited indefinitely.
4. Resource ordering prevented circular wait.
5. The prevention program executed successfully without deadlock.

Result:

Successfully demonstrated a deadlock scenario using POSIX threads, identified the four necessary conditions for deadlock, and prevented deadlock using resource ordering.